

Operating systems

Lecture #5

Content Lecture#4

01 Scheduling Algorithm

02 Scheduling Philosphy

03 Scheduling Criteria

04 Scheduling Queue

05 Scheuling

Lets take a Break



How does batman schedule a task on his computer?
With a .bat script


hello.bat


ip.bat

Revising Previous Lecture

Important System Calls Used in OS

fork()
Processes use this system call to create processes that are a copy of themselves. With the help of this system Call parent process creates a child process, and the execution of the parent process will be suspended till the child process executes.

exec()
This system call runs when an executable file in the context of an already running process that replaces the older executable file. However, the original process identifier remains as a new process is not built, but stack, data, head, data, etc. are replaced by the new process.

exit()
The exit() system call is used to terminate program execution. Specially in the multi-threaded environment, this call defines that the thread execution is complete. The OS reclaims resources that were used by the process after the use of exit() system call.

kill():
The kill() system call is used by OS to send a termination signal to a process that urges the process to exit. However, a kill system call does not necessarily mean killing the process and can have various meanings.

wait()
In some systems, a process needs to wait for another process to complete its execution. This type of situation occurs when a parent process creates a child process, and the execution of the parent process remains suspended until its child process executes.
The suspension of the parent process automatically occurs with a wait() system call. When the child process ends execution, the control moves back to the parent process.



I

Cooperating

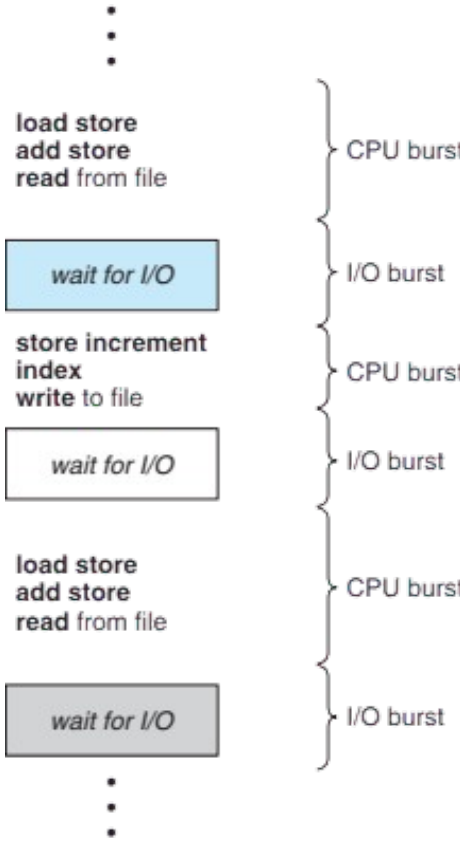
F

A

Scheduling Algorithm Objectives

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait
- CPU burst followed by I/O burst
- CPU burst distribution is of main concern



Scheduling Philosophies

Preemptive

- The system may preempt the CPU from a process at any time;
- Prevents any one process from using the CPU for too long
- Can lead to race condition (require synchronization techniques)

Non-preemptive

- Each process voluntarily gives up the CPU
- It is;
  - Simple
  - Easy to implement
  - Not suitable for multi-user systems
- If the running process become blocked next process can be scheduled

Scheduling Criteria

- CPU utilization
  keep the CPU as busy as possible
- Throughput
  # of processes that complete their execution per time unit
- Tunaround time
  amount of time to execute a particular process
- Waiting time
  amount of time a process has been waiting in the ready queue
- Response time
  amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)
- Priority
  Priority assigned to that process or job
- Class
  Class of job, i.e. batch or on-line or real-time
- Resource requirements
  E.g. expected run time, memory required, etc
- I/O or CPU bound
  i.e. whether job is I/O bound or CPU bound
- Resources used to date
  i.e. the amount of processor time already consumed
- Waiting time to date
  i.e. the amount of time spent waiting for service so far

Process Scheduling Queues

- Job queue

  Set of all the processes in the system

  It also includes the processes from the queues given below

- Ready queue

  Set of all the processes residing in main memory (ready or waiting)

- Device queue

  Set of processes waiting for an I/O devices

CPU Scheduler

- Short-term scheduler selects from among the processes in ready queue, and allocates the CPU to one of them
  - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
  - Switches from running to waiting state
  - Switches from running to ready state
  - Switches from waiting to ready
  - Terminates
- Scheduling under 1 and 4 is non-preemptive
- All other scheduling is preemptive
  - Consider access to shared data
  - Consider preemption while in kernel mode
  - Consider interrupts occurring during crucial OS activities

Levels of Scheduling

Scheduling can be exercised at three levels

- High-level scheduling (Long-term scheduler)
  - Allow limited number of processes in the ready queue to compete
  - It disallows processes beyond a certain limit for batch processes first and in the end also the interactive processes
  - Invoke less frequently
  - May be slow
- Medium-level scheduling (Medium-term scheduler)
  - It is concerned with decision to temporarily remove a process from the system (to reduce system load)
  - To reintroduce a process or swapping in
  - What if there is room in memory and both a new process as well as swapped out process want to be loaded
    - In this case, medium level scheduler has to work in close conjunction with high level scheduler

Levels of Scheduling

- Low level-scheduling (Short-term scheduler)

  Handles the decisions of which process is to be assigned to the processor (dispatcher)

  Invoke very frequently

  Must execute fast

  Its scheduling could be preemptive or non-preemptive

  Dispatch latency should be minimum

  Dispatch latency is the time it takes for the dispatcher to stop one process and start another running

  Example:

  Assume a process runs for 90ms and scheduler run for 10ms

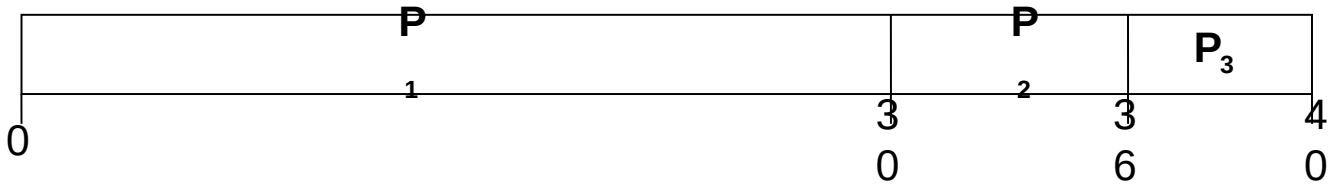
  Overheads = 10/(90+10) = 10%

- Any OS may use one or all of these levels, depending upon the desired
- These levels have to interact amongst themselves closely to ensure the computing resources are managed optimally
- The exact algorithms for these and interaction between them are quite complex

First Come First Serve Scheduling

The FCFS simply assign the processor to the process which is first in ready queue
When the processor is free, the next process at the head of the ready queue will be selected
Consider the following set of three processes that arrive at time ‘0’

Process ID	Priority	Burst Time
P1	2	30 ms
P2	3	6 ms
P3	1	4 ms



The waiting time for each process is P1=0, P2=30, P3=36
The average waiting time is (0+30+36)/3 = 22ms
The average waiting time of FCFS may vary substantially if the process’s CPU burst time vary greatly

First Come First Serve Scheduling

- Salient points of FCFS
  - It is the simplest scheduling algorithm
  - Also know as the first in first out (FIFO)
  - FCFS is non-preemptive
  - It performs much better for long processes than short processes
  - It tend to favor CPU-bond processes over I/O bound
  - It is not suitable for time sharing system
  - FCFS is rarely used on its own but is effective if combined with other scheme

Shortest Job First Scheduling

- This algorithm associates with shortest job first
- When the CPU is available, it is assigned to the process that has smallest CPU burst time
- If two processes have same CPU-burst then FCFS scheduling is used along with this to break the tie
- Moving a short process before a long one
  - Decrease the waiting time of short process
  - Increase the waiting time of long process
  - Consequently, the average time decreases
- This will result in a smaller number of PCB’s in the ready or blocked queues
- The search time will be smaller, thus improving the response time
- The number of satisfied user will increase
- A long job in the queue may delayed indefinitely by a succession of smaller jobs arriving in ready queue
- This can be avoided by setting higher external priority to those important jobs
- The OS at any time can calculates a resultant priority based on both
- Also known as shortest job next

Shortest Job First Scheduling

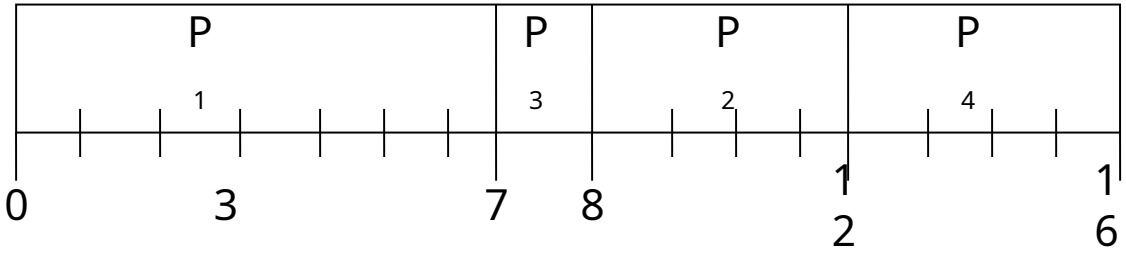
- Salient Points of SJF
  - SJF may be preemptive or non-preemptive
  - It will preempt the currently executing process if its remaining time is greater than that of a newly arrived process
  - This scheme is known as shortest remaining time first
  - SJF is optimal in sense that it yields the smallest average waiting time
  - In general, it is difficult to predict the CPU time requirement for a process
  - SJF is optimal for batch jobs for which the run times are known in advance
  - SJF reduces average waiting time over FCFS

Shortest Job First Scheduling

- **Non-Preemptive SJF Scheduling**

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)



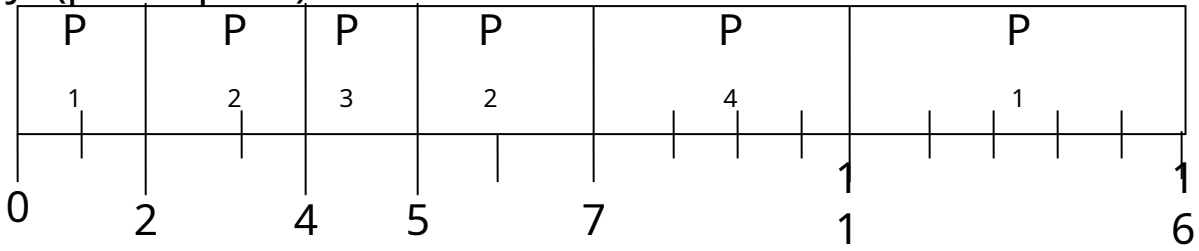
- Average waiting time = (0 + 6 + 3 + 7)/4 = 4

Shortest Job First Scheduling

- **Preemptive SJF Scheduling**

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Average waiting time = (9 + 1 + 0 +2)/4 = 3

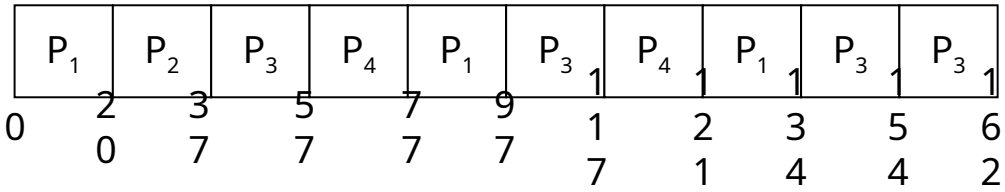
Round Robin Scheduling

- The Round-Robin (RR) scheduling is designed especially for time sharing systems
- In this algorithm there is small unit of time known as time quantum
- A time quantum is generally up to 10-100ms
- CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 quantum
- For RR scheduling we keep the ready queue as a FIFO queue of processes
- The CPU picks up the first process from the ready queue, set a timer to interrupt after 1 quantum
- If the process burst time is less than 1 quantum
    - then the process itself release the CPU voluntarily & proceed to next process
- If the process burst time is greater then 1 quantum
    - then that process is sent back to the ready queue and CPU scheduler selects the next process in the ready queue for scheduling

Round Robin Scheduling

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

- Time quantum = 20 ms
- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better response

Priority Scheduling

In priority scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority

Equal priority processes are scheduled in FCFS order

Priorities are generally some fixed range of numbers, such as 0, 1, 5, 10

There is no general agreement on whether ‘0’ is highest or lower priority

Priority can be defined on the bases of following

- Process time limit
- Memory requirements
- Number of open files
- Ratio of average I/O burst to CPU burst

Priority mechanism can be static or dynamic

- Static priority mechanism is easy to implement and have relatively low overheads
- Dynamic priority mechanism is more complex to implement and have greater overheads

The priority scheduling may be preemptive or non-preemptive

- The choice arise when a high priority process comes in the queue while a low priority process is executing

A process that is ready to run but lacking the CPU can be considered blocked

Solution to indefinite blocking is aging

Aging is a technique of gradually increasing the priority of processes that waits for a long time

OS allows only a limited priority classes

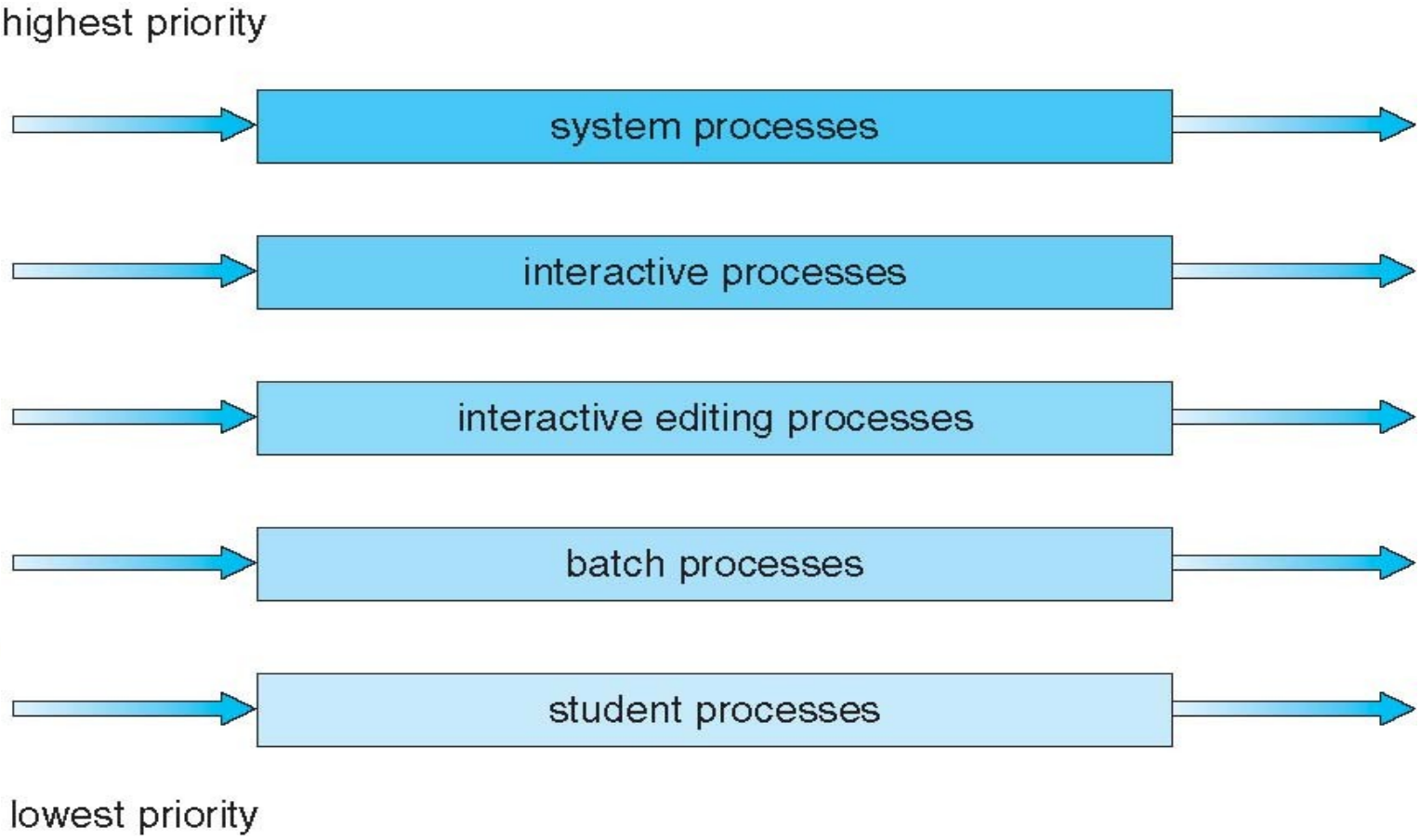
In VAX / VMS priority range is 0 to 31

In Unix priority range is -20 to 20

Multilevel Queue

- Ready queue is partitioned into separate queues:
  foreground (interactive)
  background (batch)
- Each queue has its own scheduling algorithm,
  - foreground – RR
  - background – FCFS
- Scheduling must be done between the queues
  - Fixed priority scheduling; (i.e., serve all from foreground then from background)
    - Possibility of starvation
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
  - 20% to background in FCFS

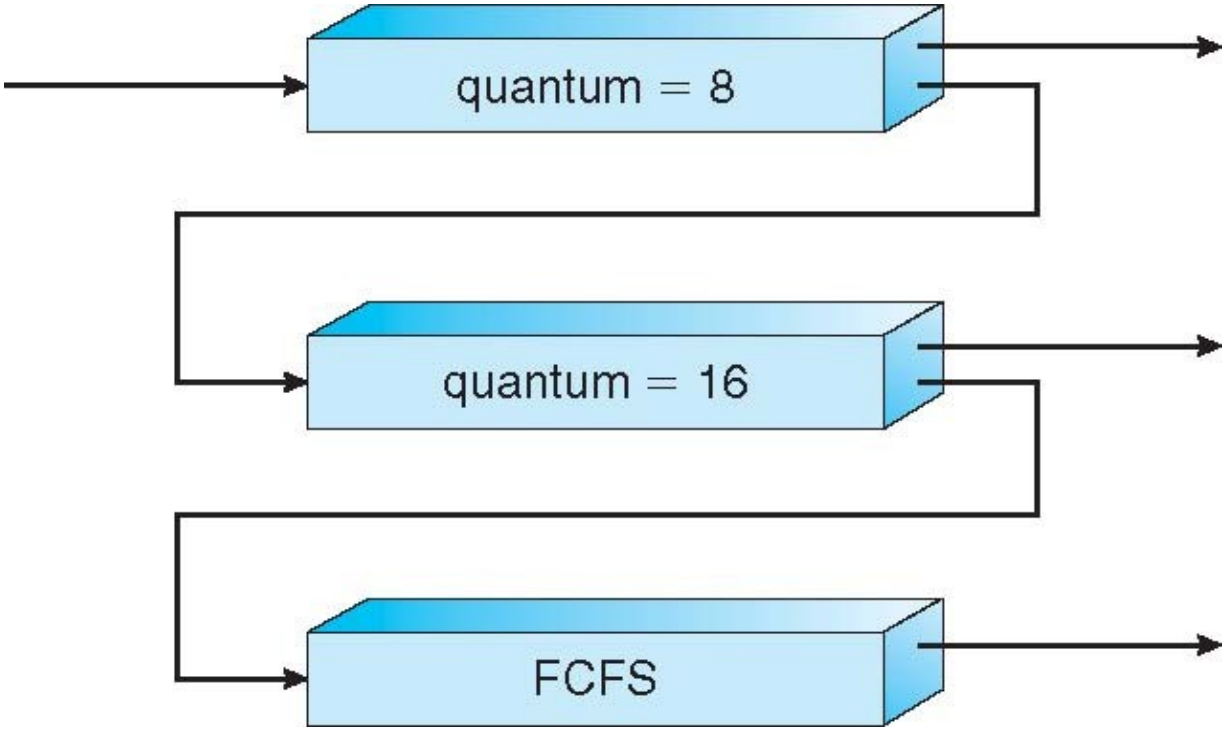
Multilevel Queue Scheduling



Multilevel Feedback Queue

- A process can move between the various queues
  - aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service
  - **Example**
- Three queues:
  - Q_0 – time quantum 8 milliseconds
  - Q_1 – time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS
    - When it gains CPU, job receives 8 milliseconds
    - If it does not finish in 8 milliseconds, job is moved to queue Q_1
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds
    - If it still does not complete, it is preempted and moved to queue Q_2

Multilevel Feedback Queues



Lab

- Run exec and file descriptor



Thank you